

Aeternum-core Smart Contract Audit

Internal Security Audit Report — Revision 2

Contract:	AeternumVault.sol
Repository:	github.com/Aeternumlabs/aeternum-core
Prepared By:	Ndubuisi Ugwuja — Founder, Aeternum Labs
Date:	3rd June 2026
Audit Type:	Internal Pre-Audit Security Assessment
Status:	Complete — External audit pending pre-mainnet

REVISION NOTE:

This document supersedes the internal audit dated 4th May 2026. The contract has been refactored from a subscription-based model to a fee-free model. All subscription, treasury, and tiering logic has been removed. A cursor-based rolling scan architecture has been introduced for scalable registry traversal. All findings, test results, and coverage figures reflect the current contract state.

Table of Contents

1. Executive Summary
2. Project Goals
3. Contract Architecture and Trust Model
4. Automated Testing
5. Static Analysis — Slither
6. Linting — Solhint
7. Detailed Findings
8. Codebase Maturity Evaluation
9. Known Limitations and Out of Scope
10. Tool Versions

1. Executive Summary

Engagement Overview

AeternumVault is a trustless, smart wallet vault with built-in automated ETH recovery powered by Chainlink Automation. Users deposit ETH into a self-sovereign vault, configure a backup address and inactivity period, and if they ever go permanently inactive — lost keys, death, incapacitation — Chainlink Automation transfers their ETH to the backup address automatically. No custodians. No admin backdoors. No subscription fees. Just code.

This internal assessment is a revision of the original May 2026 pre-audit review, updated to reflect the transition from a subscription/treasury model to a zero-fee model. The revised contract removes all fee collection, subscription tiers, and treasury management logic, and introduces a cursor-based rolling scan architecture for scalable registry traversal by Chainlink Automation.

The review employed:

- Static analysis using Slither v0.11.4
- Linting with Solhint
- Manual line-by-line code review
- 135-test Foundry unit test suite achieving 100% line and function coverage on AeternumVault.sol
- Foundry fuzz and invariant tests (128,000+ invariant calls)
- Echidna property-based fuzzing across 11 invariant properties with 50,057 randomised call sequences

No High severity vulnerabilities were identified.

Finding Summary

Severity	Count	Status
High	0	—
Medium	1	Acknowledged (mitigated)
Low	1	Acknowledged (mitigated)
Informational	3	Acknowledged

2. Project Goals

This assessment was scoped to provide a thorough internal security review of the revised AeternumVault contract. The following questions were answered:

- Does the vault correctly enforce user-specific balance isolation, preventing any cross-user fund access?
- Are all ETH-transferring code paths protected by the Checks-Effects-Interactions (CEI) pattern and ReentrancyGuard?
- Is the Chainlink Automation integration safe — does checkUpkeep correctly identify due wallets, and does performUpkeep re-validate before acting?
- Does the new cursor-based rolling scan advance correctly, hold on active recovery windows, and wrap at the end of the registry?
- Can stale performData from checkUpkeep cause incorrect recoveries or fund loss?
- Does the swap-and-pop registry removal maintain array integrity under concurrent batch removals?
- Does the abandonment mechanism after MAX_RECOVERY_ATTEMPTS prevent infinite retry loops that drain Chainlink LINK?
- Are all access control paths — onlyRegistered modifier and the forwarder address check — enforced correctly?
- Does the contract correctly reject direct ETH transfers to prevent accidental fund lock?
- Do all 11 Echidna invariant properties hold across 50,000+ randomised call sequences?
- Are there any high-severity static analysis findings not previously addressed?

3. Contract Architecture and Trust Model

AeternumVault is a single-contract architecture. There is no subscription model, no treasury, and no fee collection of any kind. Each registered wallet has its own isolated vault within the contract — balances are tracked individually per user via a private mapping, never pooled. Chainlink Automation nodes monitor all registered vaults off-chain and execute recovery on-chain only when inactivity conditions are met.

Trust Model

Actor	Can Do	Cannot Do
User	Register, deposit, send, withdrawAll, ping, update config, cancel recovery	Access other users' funds
Chainlink Automation	Call performUpkeep when inactivity conditions are met	Alter user configs or redirect funds
Anyone	Call checkUpkeep (read-only, off-chain simulation)	Trigger recovery before the inactivity period elapses
No one	—	Pause recovery, upgrade the contract, collect fees, or access user funds

Key Immutables (Set at Deployment)

Immutable	Description and Recommended Value
<code>MIN_INACTIVITY_PERIOD</code>	Minimum inactivity period in seconds. Enforced on register() and updateInactivityPeriod().
<code>MAX_INACTIVITY_PERIOD</code>	Hard ceiling on inactivity period — prevents permanent lock-up. Recommended: 10 years (3650 days).
<code>MAX_CHECK_UPKEEP_SIZE</code>	Maximum wallets scanned per checkUpkeep. Runs entirely off-chain — no block gas limit constraint. Recommended: 5000.
<code>MAX_PERFORM_UPKEEP_SIZE</code>	Maximum wallets recovered per performUpkeep. On-chain, gas-bound. Recommended: 50 on mainnet (~5M gas worst-case).

Immutable	Description and Recommended Value
<code>MAX_RECOVERY_ATTEMPTS</code>	Consecutive failed recovery attempts before a vault is permanently abandoned. Recommended: 3.
<code>CURSOR_ADVANCE_INTERVAL</code>	Seconds between idle cursor advances. Recommended: 1 hour on mainnet (24 on-chain calls/day).

RecoveryConfig Struct

Fields: backupAddress, inactivityPeriod, lastActivity, balance, isActive, failedRecoveryAttempts, isAbandoned.

Note: The tier and subscriptionExpiry fields present in the May 2026 revision have been removed as part of the transition to a fee-free architecture.

Core Components

User Vault (per address)

Each user registers with a backup address and inactivity period. Deposited ETH is held under their address key in the contract. The onlyRegistered modifier admits both isActive and isAbandoned wallets, ensuring abandoned wallet owners retain full access to their remaining balance via withdrawAll() or send().

Chainlink Automation — Cursor-Based Rolling Scan

checkUpkeep() is called off-chain by Chainlink nodes. It scans a window of up to MAX_CHECK_UPKEEP_SIZE wallets beginning at s_checkCursor. If due wallets are found, they are returned as candidates (capped at MAX_PERFORM_UPKEEP_SIZE) and the cursor holds its current position until the window is fully cleared. If no wallets are due and the registry fits in a single window, no upkeep is triggered. If the registry spans multiple windows and CURSOR_ADVANCE_INTERVAL has elapsed, an idle advance is signalled.

performUpkeep() receives ABI-encoded (address[] candidates, uint256 nextCursor, bool isIdleAdvance). It re-validates all candidates before acting, writes nextCursor to s_checkCursor unconditionally, and updates s_lastCursorAdvance only when isIdleAdvance is true — not during active recovery passes where the cursor holds position.

Registry

`s_registeredWallets` (`address[]`) tracks all active vaults. `s_walletIndexPlusOne` provides $O(1)$ position lookup via a 1-indexed mapping, enabling safe swap-and-pop removal during batch operations without corrupting array integrity across multiple removals in the same `performUpkeep` call.

Abandonment Mechanism

After `MAX_RECOVERY_ATTEMPTS` consecutive failed ETH transfers to a backup address, the vault is permanently marked `isAbandoned = true` and removed from the monitoring registry. The user retains full access to their balance via `withdrawAll()` or `send()`. Re-registration using the same backup address is permanently blocked via the `s_abandonedBackupAddresses` mapping.

4. Automated Testing

4.1 Unit Test Coverage (Foundry)

Metric	Result
Line Coverage (AeternumVault.sol)	100.00% (203/203)
Statement Coverage	98.79% (245/248)
Branch Coverage	93.62% (44/47)
Function Coverage	100.00% (26/26)
Total Tests	135 passing, 0 failing

Coverage figures above are for `AeternumVault.sol`. Deployment scripts (`Deploy.s.sol`, `HelperConfig.s.sol`) and the Echidna harness (`AeternumVaultEchidna.sol`) show 0% coverage as expected — scripts are tested by live execution, not unit tests. Mock contracts show partial coverage reflecting only the paths exercised during security testing.

Test sections covered: Constructor and deployment validation, Registration (standard and re-registration after abandonment), Deposit, Send, WithdrawAll, Ping, Config updates (backup address, inactivity period), Cancel recovery, checkUpkeep (cursor behavior, window scanning,

idle advance logic, single-window mode, edge cases), performUpkeep (execution, batch recovery, stale data handling, failure and retry, abandonment after max attempts, forwarder access control), Registry integrity (swap-and-pop batch removal), Reentrancy defense, View helpers, Fuzz tests, Invariant tests.

4.2 Fuzz Tests

Four fuzz test functions were run with 256 runs each:

- testFuzz_register_validInactivityPeriod — any period within [MIN_INACTIVITY_PERIOD, MAX_INACTIVITY_PERIOD] accepted
- testFuzz_deposit_amount — any non-zero deposit amount handled correctly
- testFuzz_send_amount — partial sends preserve balance invariants
- testFuzz_isRecoveryDue_timing — recovery only triggers at or after the inactivity period

All passed.

4.3 Foundry Invariant Tests

ID	Description	Calls	Result
FUND-INV-1	Contract ETH balance >= sum of all user vault balances	128,000	PASS
FUND-INV-2	Every wallet in the registry has isActive == true	128,000	PASS

Note: The fee-isolation invariant (accumulated fees never exceed contract ETH balance) present in the May 2026 revision has been removed. The current model carries no fees or treasury balance.

4.4 Echidna Property-Based Fuzzing

Configuration — testMode: property, testLimit: 50,000, seqLen: 100, shrinkLimit: 5,000.

Result: 50,057 calls executed. All 11 properties passing. Unique instructions: 4,089. Corpus size: 6 sequences saved. Seed: 3611586492036286283.

Property	Description	Result
----------	-------------	--------

Property	Description	Result
PROP-01	Contract ETH balance covers sum of all user vault balances	PASS
PROP-02	Every wallet in the registry has isActive == true	PASS
PROP-03	No wallet in the registry has isAbandoned == true	PASS
PROP-04	getTotalRegistered() equals the actual registry array length	PASS
PROP-05	isActive and isAbandoned are mutually exclusive per wallet	PASS
PROP-06	Every registered wallet has a valid non-zero backup address	PASS
PROP-07	No wallet with zero balance is flagged as recovery due	PASS
PROP-08	Inactivity period never exceeds MAX_INACTIVITY_PERIOD	PASS
PROP-09	failedRecoveryAttempts never exceeds MAX_RECOVERY_ATTEMPTS	PASS
PROP-10	Abandoned wallets are not present in the registry	PASS
PROP-11	Wallet inactivity period is always at or above MIN_INACTIVITY_PERIOD	PASS

Note: Three properties from the May 2026 revision (treasury address validation, Free-tier subscriptionExpiry always zero, and accumulated fees within contract balance) have been removed. The corresponding contract logic no longer exists.

5. Static Analysis — Slither

Slither v0.11.4 was run against AeternumVault.sol. The run returned 34 results across 5 contracts and 101 detectors. Findings originating exclusively from third-party library code (OpenZeppelin StorageSlot assembly) are noted as library findings and are outside the project's control. Compared to the May 2026 revision, the reentrancy finding has been reclassified by Slither from reentrancy-eth (Medium) to reentrancy-benign (Low), reflecting that config.balance is provably zeroed before the external call.

Slither Finding Summary

ID	Detector	Severity	Source	Action
S01	reentrancy-benign in _executeRecovery	Low	Contract	Acknowledged — mitigated by layers
S02	calls-loop in performUpkeep	Medium	Contract	Acknowledged — architecturally unavoidable
S03	block.timestamp comparisons in checkUpkeep	Low	Contract	Acceptable — 12-sec variance irrelevant
S04	costly-loop (pop + delete) in _removeFromRegistry	Informational	Contract	Unavoidable — swap-and-pop is optimal
S05	Inline assembly in checkUpkeep	Informational	Contract	Intentional — mstore array length trim
S06	Low-level .call{value}{}() in 4 functions	Informational	Contract	Correct modern pattern
S07	events-maths: s_checkCursor update missing event	Informational	Contract	Acknowledged — internal variable

ID	Detector	Severity	Source	Action
S08	OZ ReentrancyGuard.sol uses ^0.8.20 pragma	Informational	Library	AeternumVault pinned to 0.8.24
S09	naming-convention: public immutables not in mixedCase	Informational	Contract	Intentional — project convention

S-01 — Reentrancy-benign in `_executeRecovery`

On the failure-restore path, state variables (`config.balance` restore, `config.isActive` restore, `s_registeredWallets` push, `s_walletIndexPlusOne`, `s_abandonedBackupAddresses`) are written after the external ETH transfer call `backupAddress.call{value: amount}()`. These post-call writes are unavoidable. Slither now classifies this as reentrancy-benign — down from reentrancy-eth in the May 2026 report — because `config.balance` is provably zeroed before the call.

Three independent layers prevent exploitation:

1. `nonReentrant` on `performUpkeep()` blocks all reentrant calls at the outer function level.
2. `config.balance` is zeroed BEFORE the external call — even a successful reentrancy attempt cannot double-spend.
3. After `MAX_RECOVERY_ATTEMPTS` consecutive failures, the vault is permanently abandoned and removed from the registry.

S-02 — Calls Inside Loop

Batch ETH recovery to distinct backup addresses inherently requires a loop containing external calls. There is no architectural alternative that preserves automatic recovery without beneficiary action. Mitigations: `nonReentrant` on `performUpkeep`, CEI enforced per wallet before its ETH transfer, per-wallet re-validation in `_executeRecovery`, `MAX_PERFORM_UPKEEP_SIZE` gas cap, and graceful failure handling that does not revert the batch.

6. Linting — Solhint

Solhint was run with a custom `.solhint.json` configuration tuned to the project's conventions. `solhint src/AeternumVault.sol` returns 0 errors and 0 warnings.

Rule	Result	Notes
compiler-version	Pass	Pragma pinned to ^0.8.24 across all project files
var-name-mixedcase	Off	s_ prefix and UPPER_SNAKE_CASE for public immutables are conventions
import-path-check	Off	OZ remapping handled in foundry.toml
function-max-lines	Off	Some complex functions intentionally exceed 50 lines for readability
gas-strict-inequalities	Off	Non-strict inequalities are intentional throughout
use-natspec @param / @notice	Pass	Full @param, @notice, and @dev coverage across all functions

7. Detailed Findings

AV-01 — Reentrancy-benign in _executeRecovery

Severity	Low
Status	Acknowledged — mitigated
Category	Reentrancy
Target	AeternumVault._executeRecovery()

Description

On the failure-restore path, several state variables are written after the external ETH transfer call `backupAddress.call{value: amount}()`. These post-call writes are unavoidable — the contract can only determine a transfer has failed after making it. Slither classifies this as reentrancy-benign in this revision (downgraded from reentrancy-eth in May 2026), because `config.balance` is fully zeroed before the call and no double-spend is achievable.

Why it is safe / Mitigations

- `nonReentrant` on `performUpkeep()` blocks all reentrant calls at the outer function level.
- `config.balance` is zeroed BEFORE the external call — even a successful reentrancy attack cannot steal funds.
- After `MAX_RECOVERY_ATTEMPTS` consecutive failures, the vault is permanently abandoned and removed from the registry, eliminating the retry loop.

Resolution

The post-call writes on the failure-restore path are architecturally unavoidable. The `// slither-disable-next-line` annotation is retained with full NatSpec rationale documenting all three safety layers. The pull-payment alternative was evaluated and rejected: it would require backup addresses to manually claim funds, breaking the core product guarantee of automatic recovery without beneficiary action.

AV-02 — External Calls Inside `performUpkeep` Loop

Severity	Medium
Status	Acknowledged — documented
Category	Denial of Service Risk / Gas Usage
Target	<code>AeternumVault.performUpkeep()</code>

Description

Slither's calls-loop detector flags the ETH transfer to `backupAddress` inside the `performUpkeep` recovery loop. This is an inherent structural requirement of a batch automated recovery system — each wallet's funds must be transferred to a distinct backup address within the same upkeep call.

Why it is safe / Mitigations

- `nonReentrant` on `performUpkeep` prevents reentrant exploitation through any backup address callback.
- CEI pattern enforced — all state changes for each wallet complete before its ETH transfer in every loop iteration.
- `_executeRecovery` re-validates each wallet individually before acting, preventing stale data exploitation.
- `MAX_PERFORM_UPKEEP_SIZE` caps loop iterations to remain within safe on-chain block gas limits.
- Failed transfers trigger the failure-restore mechanism, not a batch revert — the batch continues for remaining wallets.

Resolution

Acknowledged and documented in the performUpkeep NatSpec @dev block with full rationale.

AV-03 — Missing State Change Event for s_checkCursor Update

Severity	Informational
Status	Acknowledged
Category	Event Emission / Observability
Target	AeternumVault.performUpkeep() — s_checkCursor = nextCursor

Description

Slither's events-maths detector flags that the assignment `s_checkCursor = nextCursor` in `performUpkeep` does not emit a corresponding event. The cursor is an internal pagination control variable — its value at any point is deterministically derivable from wallet-level events (`RecoveryExecuted`, `RecoveryFailed`, `RecoveryAbandoned`) emitted during the same transaction.

Why it is safe / Mitigations

- The cursor does not represent a user-facing value or financial state.
- Emitting a `CursorAdvanced` event on every `performUpkeep` call would add unnecessary gas cost without providing security transparency beyond what is already derivable.
- The off-chain watcher service and frontend can track cursor position through the `getCheckCursor()` and `getLastCursorAdvance()` view functions.

Resolution

Acknowledged. A dedicated cursor event may be added in a future revision if monitoring tooling requires it. No security risk is present.

AV-04 — setForwarder() Has No Caller Restriction

Severity	Informational
Status	Acknowledged — operational guidance documented
Category	Access Control / Operational Risk
Target	AeternumVault.setForwarder()

Description

`setForwarder()` is callable by any address before the forwarder has been set. If an attacker calls it with an address they control before the legitimate operator can, `performUpkeep()` will be permanently locked to that attacker-controlled forwarder address, preventing Chainlink Automation from executing recoveries.

Why it is safe / Mitigations

- The attack window is narrow: `setForwarder()` should be called in the same deployment sequence immediately after upkeep registration.
- User funds are never at direct risk — onlyRegistered functions (`withdrawAll`, `send`, `cancelRecovery`) remain fully accessible.
- The permissionless design is intentional: the contract has no owner role, and introducing one solely to restrict `setForwarder()` would undermine the trustless architecture.

Resolution

Acknowledged as an operational risk. Deployment documentation will instruct operators to call `setForwarder()` immediately after upkeep registration. No code change is planned given the trustless design constraint.

AV-05 — OpenZeppelin Dependency Uses ^0.8.20 Pragma

Severity	Informational
Status	Acknowledged
Category	Configuration
Target	lib/openzeppelin-contracts/contracts/utils/ReentrancyGuard.sol

Description

OpenZeppelin's `ReentrancyGuard.sol` and `StorageSlot.sol` specify `pragma solidity ^0.8.20`, which Slither flags as containing known compiler issues: `VerbatimInvalidDeduplication`, `FullInlinerNonExpressionSplitArgumentEvaluationOrder`, and `MissingSideEffectsOnSelectorAccess`.

Why it is safe / Mitigations

- `AeternumVault.sol` is pinned to `pragma solidity ^0.8.24`, which resolves all three cited issues.
- The OZ files are third-party dependencies compiled under the project's pinned compiler version and cannot be modified.

Resolution

Acknowledged. No action required.

8. Codebase Maturity Evaluation

Category	Rating	Notes
----------	--------	-------

Category	Rating	Notes
Arithmetic	Satisfactory	Solidity ^0.8.24 provides native overflow protection. unchecked{} used only in bounded loop counters.
Auditing & Logging	Satisfactory	12 events covering all state-changing operations: RecoveryRegistered, Deposited, Sent, Withdrawn, RecoveryExecuted, RecoveryFailed, RecoveryAbandoned, RecoveryCancelled, ActivityPinged, BackupAddressUpdated, InactivityPeriodUpdated, ForwarderSet.
Access Controls	Strong	onlyRegistered modifier gates all user-facing state-changing functions, correctly admitting both isActive and isAbandoned wallets. No treasury, no admin, no owner role. setForwarder() is permissionless by design — one-time-set, operational guidance documented.
Complexity Management	Satisfactory	Clean separation: user functions, Chainlink Automation functions (checkUpkeep, performUpkeep), internal helpers (_executeRecovery, _removeFromRegistry). Interface fully separates type definitions from implementation.
Decentralisation	Strong	No upgradeability. No admin pause. No owner override. No fee collection of any kind. Trustless by design — only the user and pre-authorised backup address can ever receive funds.

Category	Rating	Notes
Documentation	Satisfactory	Comprehensive NatSpec with full @param, @notice, and @dev coverage across all functions. Trust model, CEI pattern, cursor architecture, and security considerations documented inline.
Low-Level Manipulation	Satisfactory	Single inline assembly block in checkUpkeep for array length trimming (mstore). Justified for correctness and gas efficiency. Documented and suppressed via Slither comment.
Testing & Verification	Strong	135 unit tests, 4 fuzz tests, 2 Foundry invariants (128,000+ calls), 11 Echidna properties (50,057 calls), Slither, Solhint. 100% line and function coverage on AeternumVault.sol. Contract deployed and functioning on Sepolia testnet.
Reentrancy	Satisfactory	ReentrancyGuard on all ETH-transferring functions. CEI enforced throughout. Post-call writes on the failure-restore path are bounded, non-exploitable for fund theft, and classified as reentrancy-benign by Slither.
Transaction Ordering	Satisfactory	performUpkeep re-validates each wallet before acting. Stale checkUpkeep data is silently skipped. No flash loan or frontrunning attack surface identified for user funds.

9. Known Limitations and Out of Scope

The following items are known limitations of this internal assessment and should be addressed by an external professional audit prior to mainnet deployment:

- The current version supports native ETH only. ERC-20 token support is planned for upcoming phase.
- The frontend application (Next.js) is outside the scope of this contract audit and requires separate security review.
- The Ponder indexer and PostgreSQL database layer used for activity history storage are outside the scope of this contract audit and require separate infrastructure and data security review.
- This assessment was conducted by the contract author. An independent external audit is required before mainnet deployment.
- Chainlink Automation upkeep LINK funding levels and top-up strategy have not been formally modelled for production traffic volumes.
- Mainnet gas cost estimates have not been validated under production fork conditions.
- The Echidna fuzzing corpus covers 50,057 calls across 11 properties. Longer runs with higher testLimit values are recommended before external audit engagement.
- The contract does not implement ERC-4337 account abstraction. This is a deliberate design decision, as EIP-7702 hybrid account integration is planned for Phase 2.

10. Tool Versions

Tool	Version	Purpose
Foundry forge	Latest stable (June 2026)	Unit, fuzz, and invariant testing
Slither	0.11.4	Static analysis
Solhint	Latest stable	Solidity linting

Tool	Version	Purpose
Echidna	Latest (Docker — ghcr.io/crytic/echidna)	Property-based fuzzing
Solc	0.8.24	Solidity compiler
OpenZeppelin Contracts	v5.x	ReentrancyGuard dependency

This internal audit report was prepared by Ndubuisi Ugwuja (Aeternum Labs) as a revised pre-audit security assessment. It supersedes the internal audit dated 4th May 2026. It does not constitute a professional third-party audit. An independent external audit is required before mainnet deployment.

© 2026 Aeternum Labs. All rights reserved.